



Universidade de Brasília
Departamento de Ciência da Computação

Tipo Abstrato de Dados

Professora: Eliza Gomes

E-mail: eliza.gomes@unb.br

Tipo de Dados

- ▶ Um tipo de dado define o conjunto de valores (domínio) e operações que uma variável pode assumir;
 - ✓ **Exemplo:** o tipo char da linguagem C suporta valores inteiros que vão de - 128 até + 127
- ▶ Além disso, esse tipo também suporta operações de soma, subtração, etc;
- ▶ Esses tipos de dados não possuem nenhum tipo de estrutura sobre seus valores.

Tipo de Dados

- ▶ Existem outros tipos de dados chamados **dados estruturados** ou **estruturas de dados**;
- ▶ Neles, existe uma relação estrutural (que pode ser linear ou não linear) entre seus valores;
- ▶ Uma estrutura de dados consiste de um conjunto de tipos de dados onde existe algum tipo de relacionamento lógico estrutural.

Tipo de Dados

- ▶ Uma estrutura de dados é apenas uma forma de armazenar e organizar os dados de modo que eles possam ser usados de forma eficiente;
- ▶ Alguns exemplos dessas estruturas de dados presentes na linguagem C são os **array**, **struct**, **union** e **enum**, todas criadas a partir dos tipos de dados básicos.

Tipo Abstrato de Dados

- ▶ Às vezes, os tipos de dados e as estruturas de dados presentes na linguagem podem não ser suficientes para a aplicação;
- ▶ Pode-se necessitar de uma melhor estruturação dos dados, assim como é preciso especificar quais operações estarão disponíveis para manipular esses dados;
- ▶ Nesse caso, convém criar um **tipo abstrato de dados**, também conhecido como TAD.

Tipo Abstrato de Dados

- ▶ Um tipo abstrato de dados, ou TAD, é um conjunto de dados estruturados e as operações que podem ser executadas sobre esses dados;
- ▶ Basicamente, o **tipo abstrato de dados** é um conjunto de valores com seu comportamento definido por operações implementadas na forma de funções;
- ▶ Ele é construído a partir dos tipos básicos (**int**, **char**, **float** e **double**) ou dos tipos estruturados (**array**, **struct**, **union** e **enum**) da linguagem C.

Tipo Abstrato de Dados

- ▶ Tipos abstratos de dados são entidades puramente teóricas, usadas para simplificar a descrição de algoritmos abstratos, classificar e avaliar as estruturas de dados e descrever formalmente certos tipos de sistemas;
- ▶ Tanto a representação quanto as operações do TAD são especificadas pelo programador;
- ▶ O usuário utiliza o TAD como uma **caixa-preta** por meio de sua interface.

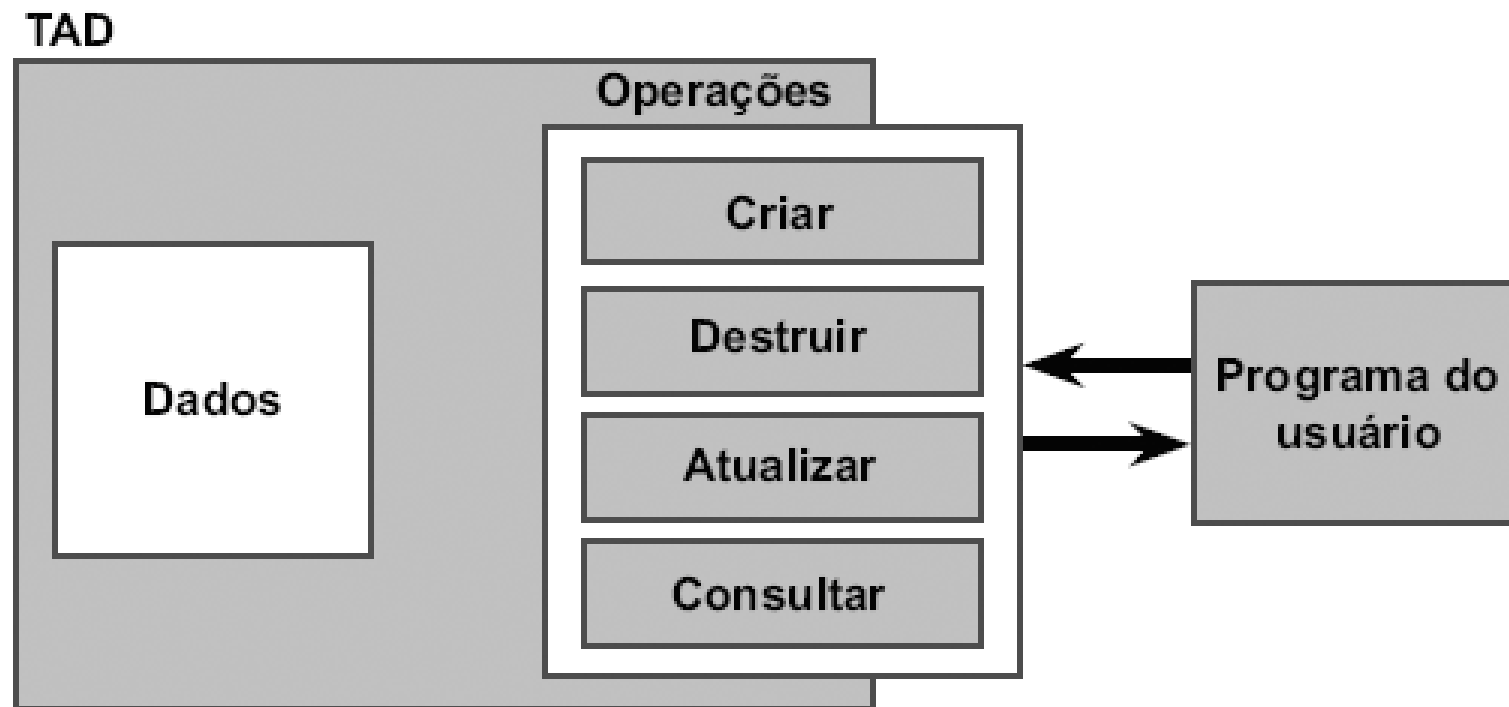
Tipo Abstrato de Dados

- ▶ Para a criação de um TAD é essencial ocultar os dados do usuário, ou seja, deve-se tornar invisível a sua implementação para o usuário;
- ▶ Assim, o TAD é como uma caixa preta para o usuário, que nunca tem acesso direto a informação lá armazenada;
- ▶ A implementação de um TAD está desvinculada de sua utilização, ou seja, quando definimos um TAD estamos preocupados com o que ele faz e não em como ele faz.

Tipo Abstrato de Dados

- ▶ Um TAD é muitas vezes implementado na forma de dois módulos:
 - ✓ Implementação; e
 - ✓ Interface
- ▶ O módulo de interface declara as funções que correspondem às operações do TAD e é visível pelo usuário;
- ▶ Essa estratégia de ocultação de informações permite a implementação e manutenção de módulos sem afetar os programas do usuário.

Tipo Abstrato de Dados



Vantagens de usar um TAD

► Encapsulamento

- ✓ Ao ocultar a implementação, é fornecido um conjunto de operações possíveis para o TAD
- ✓ Isso é tudo o que o usuário precisa saber para fazer uso do TAD
- ✓ O usuário não precisa de nenhum conhecimento técnico de como a implementação trabalha para usá-lo, tornando o seu uso muito mais fácil

► Segurança

- ✓ O usuário não tem acesso direto aos dados
- ✓ Isso evita que ele manipule os dados de maneira errada

Vantagens de usar um TAD

► Flexibilidade

- ✓ Podemos alterar o TAD sem alterar as aplicações que o utilizam
- ✓ Na verdade, podemos ter diferentes implementações de um mesmo TAD desde que todos respeitem a mesma interface
- ✓ Assim, podemos usar a implementação mais eficiente para determinada situação

► Reutilização

- ✓ A implementação da TAD é feita em um módulo diferente do programa do usuário

O tipo FILE

- ▶ Para trabalhar com arquivos na linguagem C precisamos usar um TAD;
- ▶ Ele é tipo FILE;
- ▶ O tipo FILE é uma estrutura que contém as informações sobre um arquivo ou fluxo de texto necessárias para realizar as operações de entrada ou saída sobre ele:
 - ✓ o descritor do arquivo
 - ✓ a posição atual dentro do arquivo
 - ✓ um indicador de fim de arquivo
 - ✓ um indicador de erro

O tipo FILE

```
01  typedef struct{
02      int      level;          // nível do buffer
03      unsigned flags;          // flag de status do arquivo
04      char      fd;             // descritor do arquivo
05      unsigned char hold;      // retorna caractere sem buffer
06      int      bsize;          // tamanho do buffer
07      unsigned char *buffer;   // buffer de transferência de dados
08      unsigned char *curp;     // ponteiro atualmente ativo
09      unsigned      istemp;     // indicador de arquivo temporário
10      short      token;        // usado para validação
11  }FILE;
12
```

O tipo FILE

- ▶ Ninguém em sã consciência deve fazer uso direto dos campos dessa estrutura;
- ▶ Então, a única maneira de trabalhar com arquivos em linguagem C é declarando um ponteiro de arquivo da seguinte maneira:

`FILE *f;`

- ▶ Desse modo, o usuário possui apenas um ponteiro para onde os dados estão armazenados, mas não pode acessá-los diretamente;
- ▶ A única maneira de acessar o conteúdo do ponteiro FILE é por meio das operações definidas em sua interface.

O tipo FILE

► Os dados do ponteiro `f` somente podem ser acessados pelas funções de manipulação do tipo FILE:

- ✓ `fopen()`

- ✓ `fclose()`

- ✓ `fputc()`

- ✓ `fgetc()`

- ✓ `feof()`

- ✓ etc.

Modularizando o programa

- ▶ Quando é trabalho com TAD na linguagem C, a convenção é utilizar dois arquivos para implementar;
- ▶ Dessa forma podemos separar o “conceito” (definição do tipo) de sua “implementação”:
 - ✓ Um arquivo “.H”: aqui são declarados os protótipos das funções visíveis para o usuário, tipos de ponteiro, e dados globalmente acessíveis. Aqui é definida a interface visível pelo usuário
 - ✓ Um arquivo “.C”: declaração do tipo de dados que ficará oculto do usuário do TAD e implementação das suas funções. Aqui é definido tudo que ficará oculto do usuário.

Modularizando o programa

- ▶ Chamamos modularização o processo de separação da definição do TAD em dois arquivos
- ▶ A modularização visa a criação de módulos. Um módulo é uma unidade com um propósito único e bem definido e que pode ser compilado separadamente do restante do programa
- ▶ Desse modo, um módulo pode ser facilmente reutilizado e modificado independente do programa do usuário
- ▶ Um módulo pode conter um ou mais tipos, variáveis, constantes e funções, além de uma interface apropriada com outros módulos existentes
- ▶ À medida que uma aplicação se torna maior, o uso de módulos se torna necessário.

Porquê?

Modularizando o programa

- ▶ O uso de um único arquivo causa uma série de problemas...
- ▶ A redação e modificação do programa se tornam mais difíceis à medida que ele fica maior:
 - ✓ a reutilização do código é um processo de copiar e colar
 - ✓ qualquer modificação exige a recompilação de todo o código
 - ✓ o código é dividido em arquivos separados com entidades relacionadas

Modularizando o programa

► A ideia básica da modularização é **dividir para conquistar**. Essa estratégia apresenta uma série de vantagens:

- ✓ divide-se um problema maior em um conjunto de problemas menores
- ✓ aumenta as possibilidades de reutilização do código
- ✓ facilita o entendimento do programa
- ✓ encapsulam os dados e, assim, são agrupados os dados e os processos logicamente relacionados
- ✓ o código fica distribuído em vários arquivos. Isso permite trabalhar com equipes de programadores
- ✓ aumento da produtividade do programador
- ✓ apenas as partes alteradas do programa precisam ser recompiladas
- ✓ verificação independente dos módulos antes do uso no programa do usuário
- ✓ maior confiabilidade

Exercício: TAD de um ponto no plano cartesiano

- ▶ Vamos criar um TAD que represente um ponto de duas dimensões no plano cartesiano;
- ▶ Defina a interface do ponto no arquivo *ponto.h*
 - ✓ Definição do novo tipo de dados (neste momento)
 - ✓ Definição das operações: `imprime_ponto`; `get_quadrante_ponto`; `distancia_entre_pontos`
- ▶ Defina a implementação do ponto no arquivo *ponto.c*

Resumo

- ▶ Os Tipos Abstratos de Dados (TADs) são modelos que definem dados e operações associadas, focando no “**o que faz**” e não no “**como faz**”;
- ▶ Características principais:
 - **Encapsulamento:** esconde detalhes internos e expõe apenas uma interface clara
 - **Abstração:** separa o uso da implementação
 - **Operações bem definidas:** como criar, acessar e modificar os dados
- ▶ TADs trazem modularidade, reutilização e flexibilidade, sendo a base para estruturas de dados e usados frequentemente em programação orientada a objetos.